Segundo Parcial

Proyecto de Desarrollo de una Aplicación Cliente/Servidor con Integración de Base de Datos

Propósito del segundo parcial

El segundo parcial consiste en el desarrollo de un **proyecto de software funcional bajo arquitectura cliente/servidor**, orientado a resolver una problemática real o simulada de una organización, empresa, institución o proceso productivo.

El objetivo principal es que el estudiante demuestre su capacidad para **analizar un problema, diseñar una base de datos relacional, implementar operaciones CRUD, integrar la base de datos con una aplicación y estructurar adecuadamente las diferentes capas del sistema**.

> **Importante:** no se evaluará únicamente que la aplicación "funcione". También se evaluará la calidad del diseño de la base de datos, la correcta integración entre sus módulos, la arquitectura utilizada, la organización del código, la documentación técnica y la explicación del proyecto.

1. Características generales del proyecto

El proyecto deberá cumplir como mínimo con las siguientes condiciones:

Arquitectura

Cliente/Servidor

El sistema deberá implementar una aplicación donde exista interacción entre una interfaz de usuario, la lógica del sistema y un servidor de base de datos.

Base de datos

Mínimo 12 tablas

El modelo relacional deberá contener como mínimo **12 tablas relacionadas entre sí** y justificadas según el problema seleccionado.

Persistencia

CRUD completo

La aplicación deberá implementar operaciones de **INSERT, SELECT, UPDATE y DELETE** desde sus diferentes módulos.

Evidencia

Video en YouTube

El funcionamiento del software deberá demostrarse mediante un video explicativo publicado en YouTube.

2. Planteamiento del problema

Cada grupo deberá seleccionar una problemática que pueda ser solucionada o gestionada mediante una aplicación. El sistema debe tener un propósito claramente definido y debe justificar por qué requiere almacenamiento de información en una base de datos.

Algunos ejemplos de posibles contextos son:

- Sistema de gestión para una inmobiliaria.
- Sistema de administración de un gimnasio.
- Sistema de gestión para un restaurante.
- Sistema de administración de un parqueadero.
- Sistema de administración de una agencia de viajes.
- Sistema de administración de una aseguradora.
- Sistema de gestión de un centro deportivo.
- Sistema de administración de una constructora.
- Sistema de administración de un laboratorio de análisis.
- Sistema de gestión de un conjunto residencial.
- Sistema de administración de una empresa de seguridad privada.
- Sistema de gestión para una empresa de recursos humanos.
- Sistema de administración de préstamos y créditos.
- Sistema de gestión para una empresa agrícola.
- Sistema de administración de una finca ganadera.
- Sistema de gestión de una empresa de producción audiovisual.
- Sistema de administración de un taller automotriz.
- Sistema de gestión para una empresa de alquiler de maquinaria.
- Sistema de administración de una empresa de turismo ecológico.
- Sistema de gestión de una tienda de comercio electrónico.
- Sistema de administración de una empresa de domicilios.
- Sistema de administración de una empresa de construcción de viviendas.
- Sistema de gestión para una academia deportiva.

- Sistema de administración de un centro de estética.
- Sistema de gestión de una funeraria.
- Sistema de administración de una empresa de producción de alimentos.

> El problema seleccionado debe permitir identificar claramente diferentes entidades, procesos, usuarios, operaciones e información suficiente para construir una base de datos de al menos 12 tablas.

3. Diseño y modelo de base de datos

El proyecto deberá presentar un **modelo relacional coherente, normalizado y directamente relacionado con la problemática seleccionada**.

Requisitos mínimos del modelo

- Mínimo **12 tablas**.
- Todas las tablas deben tener una razón de existir dentro del sistema.
- Cada tabla debe poseer una clave primaria.
- Las relaciones deben implementarse mediante claves foráneas.
- Deben existir relaciones 1:1, 1:N o N:M cuando el problema las requiera.
- Las relaciones muchos a muchos deberán resolverse mediante tablas intermedias.
- Los tipos de datos deben ser adecuados para cada atributo.
- Se deben utilizar restricciones de integridad cuando sean necesarias.
- El diseño debe evitar duplicación innecesaria de información.

Documentación del modelo

El grupo deberá entregar:

- Modelo relacional completo.
- Diagrama con tablas, atributos, claves primarias y claves foráneas.
- Esquema final implementado en el motor de base de datos.
- Archivo SQL de creación de la base de datos.
- Backup completo de la base de datos.

4. Arquitectura del sistema

El proyecto debe demostrar una separación clara de responsabilidades. Como mínimo, el sistema debe diferenciar las siguientes capas:

Capa de presentación

Corresponde a las interfaces mediante las cuales el usuario interactúa con el sistema.

- Formularios.
- Menús.
- Tablas de consulta.
- Paneles.
- Botones y acciones.
- Validaciones visuales.

Capa lógica

Contiene las reglas de negocio y controla el comportamiento de la aplicación.

- Validación de información.
- Procesamiento de datos.
- Reglas propias del problema.
- Comunicación entre interfaz y datos.
- Control de operaciones.

Capa física / acceso a datos

Es la responsable de almacenar, consultar y modificar la información de manera persistente.

- Servidor de base de datos.
- Conexiones.
- Consultas SQL.
- Persistencia.
- Procedimientos o servicios de acceso a datos, si son utilizados.

> El código no debe mezclar de manera desorganizada interfaz, reglas de negocio y consultas SQL. El estudiante deberá poder explicar qué componentes pertenecen a cada capa.

5. Operaciones CRUD obligatorias

La aplicación deberá realizar operaciones reales contra la base de datos. No se aceptará que los datos sean simulados, quemados directamente en el código o almacenados únicamente en variables temporales.

Operación	SQL asociado	Qué debe demostrar la aplicación
Crear	`INSERT`	Registrar nuevos datos desde formularios o módulos del sistema.
Consultar	`SELECT`	Visualizar información proveniente directamente de la base de datos.
Actualizar	`UPDATE`	Modificar registros existentes y reflejar los cambios en la base de datos.
Eliminar	`DELETE`	Eliminar registros, respetando las relaciones e integridad de la información.

> No es suficiente mostrar un único CRUD aislado. Las operaciones deben estar integradas con los módulos principales del sistema y ser coherentes con la problemática elegida.

6. Módulos funcionales

El sistema deberá dividirse en módulos funcionales. Cada grupo deberá explicar claramente:

- Cuál es el objetivo de cada módulo.
- Qué usuario o actor utiliza el módulo.
- Qué tablas de la base de datos intervienen.
- Qué operaciones realiza sobre los datos.
- Qué relaciones entre tablas utiliza.
- Cómo se conecta el módulo con otros componentes de la aplicación.

Ejemplo de explicación esperada

Módulo: Gestión de ventas.

Objetivo: registrar las ventas realizadas por los clientes.

Tablas utilizadas: cliente, venta, detalle_venta, producto y empleado.

Uso de la base de datos: consulta el cliente y los productos disponibles, registra la venta, almacena sus detalles y actualiza las cantidades disponibles.

Operaciones utilizadas: SELECT, INSERT y UPDATE.

7. Apartado de informes

El software deberá incluir obligatoriamente un módulo o apartado destinado a la generación de **informes relevantes para la problemática**.

Los informes no deben limitarse a mostrar el contenido completo de una tabla. Deben generar información útil mediante consultas, filtros, agrupaciones, cálculos, relaciones entre tablas o indicadores.

Ejemplos de informes válidos

- Ventas realizadas por período.

- Productos con mayor cantidad de ventas.
- Clientes con mayor número de compras.
- Reservas activas, canceladas y completadas.
- Servicios más solicitados.
- Ingresos mensuales.
- Cantidad de atenciones por profesional.
- Inventario disponible y productos próximos a agotarse.
- Usuarios o procesos con mayor actividad.

> Los informes deben construirse utilizando información almacenada en la base de datos y deben demostrar el uso de consultas relevantes, preferiblemente involucrando varias tablas.

8. Entregables obligatorios

El proyecto deberá entregarse en un único archivo comprimido, organizado correctamente.

- ✓ Código fuente completo de la aplicación.
- ✓ Archivo de backup de la base de datos.
- ✓ Script SQL con la estructura o esquema de la base de datos.
- ✓ Modelo relacional.
- ✓ Diagrama de la base de datos con mínimo 12 tablas.
- ✓ Documento breve con la descripción del problema.
- ✓ Descripción de los módulos desarrollados.
- ✓ Explicación de las tres capas del sistema.
- ✓ Enlace del video publicado en YouTube.
- ✓ Instrucciones necesarias para ejecutar el proyecto.

Estructura sugerida del archivo comprimido

```
SEGUNDO_PARCIAL_GRUPO_X/
|
├── codigo_fuente/
|   └── proyecto completo
|
├── base_datos/
|   ├── esquema.sql
|   └── backup_base_datos
|
├── modelos/
|   ├── modelo_relacional.pdf
|
├── documentacion/
|   └── descripcion_proyecto.pdf
|
└── enlace_video.txt
```

9. Video explicativo en YouTube

Cada grupo deberá publicar un video en YouTube y entregar el enlace correspondiente. El video deberá permitir verificar que el proyecto funciona y que los integrantes comprenden la solución desarrollada.

El video debe mostrar como mínimo:

1. Descripción breve del problema que resuelve el software.
2. Presentación del modelo relacional.
3. Explicación general de las principales tablas y relaciones.
4. Explicación de la arquitectura por capas.
5. Demostración del funcionamiento de la aplicación.
6. Creación de registros mediante INSERT.
7. Consulta de información mediante SELECT.
8. Actualización de registros mediante UPDATE.
9. Eliminación de registros mediante DELETE.
10. Explicación de los diferentes módulos.
11. Explicación de cómo cada módulo utiliza la base de datos.
12. Demostración del módulo de informes.

> Todos los integrantes del grupo deben demostrar participación y conocimiento del proyecto. El video no debe limitarse a una lectura de diapositivas.

10. Aspectos técnicos que se evaluarán

Base de datos

- Diseño correcto.
- Relaciones coherentes.
- Integridad referencial.
- Tipos de datos adecuados.
- Uso real dentro del sistema.

Programación

- Organización del código.
- Separación por capas.
- Validaciones.
- Uso correcto de la conexión.
- Implementación CRUD.

Funcionalidad

- Módulos coherentes.
- Navegación clara.
- Datos persistentes.
- Informes útiles.
- Funcionamiento estable.

Sustentación

- Dominio del proyecto.
- Claridad en las explicaciones.
- Capacidad de justificar el diseño.
- Comprensión del uso de la base de datos.

11. Rúbrica sugerida de evaluación

Criterio	Descripción	Valor
Planteamiento y solución propuesta	Claridad del problema, alcance y coherencia de la aplicación desarrollada.	**10%**

Criterio	Descripción	Valor
Modelo lógico y relacional	Mínimo 12 tablas, relaciones correctas, PK, FK, normalización y consistencia.	**20%**
Arquitectura por capas	Separación clara entre presentación, lógica y acceso físico a datos.	**15%**
CRUD e integración con base de datos	INSERT, SELECT, UPDATE y DELETE correctamente implementados desde la aplicación.	**20%**
Módulos funcionales	Coherencia, utilidad, integración y explicación de cómo utilizan la base de datos.	**10%**
Informes	Consultas relevantes que transforman datos en información útil.	**10%**
Entregables técnicos	Código, backup, script SQL, modelos, documentación y organización del comprimido.	**5%**
Video y sustentación	Claridad, demostración del sistema, participación y dominio técnico del proyecto.	**10%**
TOTAL		**100%**

12. Condiciones de entrega

- El sistema debe poder ejecutarse correctamente.
- La base de datos entregada debe corresponder con la utilizada por el software.
- El backup debe poder restaurarse.
- El enlace de YouTube debe ser accesible para el docente.
- No se deben entregar únicamente capturas de pantalla como evidencia de funcionamiento.
- El código fuente entregado debe corresponder exactamente con el sistema mostrado en el video.
- Los modelos deben coincidir con la estructura implementada en la base de datos.
- Las relaciones entre tablas deben tener una justificación funcional.

13. Criterios que pueden reducir la calificación

- Entregar menos de 12 tablas.
- Crear tablas sin propósito únicamente para completar el número mínimo.
- No implementar claves foráneas.
- Tener un modelo diferente a la base de datos realmente utilizada.
- No implementar alguna de las operaciones CRUD.
- No presentar informes.
- Usar datos simulados en lugar de consultar la base de datos.
- Mezclar completamente presentación, lógica y consultas SQL sin una estructura definida.
- Entregar un proyecto que no pueda ejecutarse.
- No entregar backup o esquema SQL.
- Video incompleto o enlace no disponible.
- No poder explicar cómo los módulos del sistema utilizan la base de datos.

14. Resultado esperado

Al finalizar el segundo parcial, el estudiante deberá demostrar que puede transformar una necesidad del mundo real en una solución de software estructurada, diseñando una base de datos relacional, desarrollando una aplicación cliente/servidor e integrando correctamente las diferentes capas de una solución informática.

La evaluación busca comprobar no solamente la capacidad de programar, sino también la capacidad para **modelar información, tomar decisiones de diseño, implementar persistencia, consultar datos, generar información útil y explicar técnicamente la solución desarrollada**.

Segundo Parcial · Proyecto Aplicación Cliente/Servidor · Evaluación Universitaria